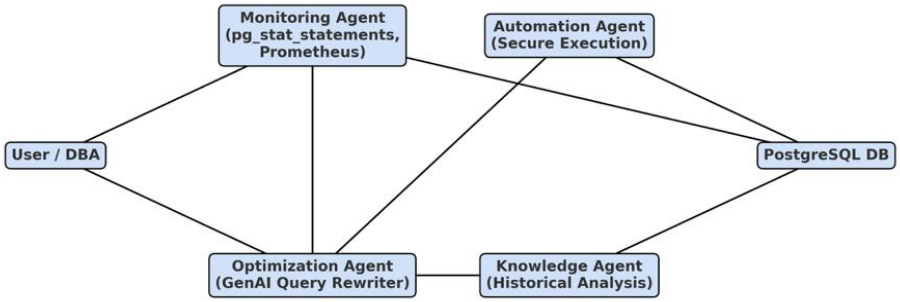

The above diagram I made is the architecture flow for your *PostgreSQL DBA Copilot* idea.

Here's the breakdown of each part:

1. User / DBA

- This is you or any database administrator interacting with the system.
- The DBA can give natural language inputs like:
- Find slow queries in the last 2 hours
- Suggest an index for query X
- Tune database for heavy read workload

2. Monitoring Agent (pg_stat_statements, Prometheus)

- Continuously collects PostgreSQL metrics → execution plans, slow query logs, wait events, locks, CPU/memory usage.
- Works with tools like **pg_stat_statements**, **EXPLAIN/ANALYZE**, and external monitoring like **Prometheus/Grafana**.
- Sends the gathered data to the **Optimization Agent**.

3. Optimization Agent (GenAI Query Rewriter)

- Uses **Generative AI** to read the monitoring data and detect inefficient queries, missing indexes, or suboptimal configurations.
- Suggests rewritten queries or database settings.
- Passes its suggestions to both the **Automation Agent** and **Knowledge Agent**.

4. Automation Agent (Secure Execution)

- Executes the approved recommendations.
- Runs only after DBA approval for sensitive changes (human-in-the-loop).
- Can apply fixes like:
- Creating indexes
- Updating PostgreSQL parameters (e.g., work_mem, shared_buffers)
- Killing blocking sessions

5. Knowledge Agent (Historical Analysis)

- Learns from past database incidents and fixes.
- Builds a knowledge base to give preventive recommendations in the future.
- Acts as the memory of the Copilot, improving over time.

6. PostgreSQL Database

- The target database where performance tuning and monitoring happen.
- All agents interact with this securely.

Flow Summary:
User → **Monitoring Agent** collects live metrics → **Optimization Agent** uses GenAI to create recommendations → either stored in **Knowledge Agent** for learning or sent to **Automation Agent** for execution → changes applied to **PostgreSQL DB**.

***